

packages/graph-core/src/index.ts

Kind: added · Baseline: a9c34bfb03fe · Target: NovaDiff · Generated: 2026-05-20T05:26:16.490Z

Overview

A new file `packages/graph-core/src/index.ts` (lines 1-126) consolidates the public surface of the `graph-core` package. It re-exports core schemas, utilities, plugins, analyzers, search, staleness, layer detection, tour generation, language lessons, plugin registry, language configs, parsers, ignore utilities, and a starter ignore file generator.

Key changes

- `export *` from `"./types.js"` and ``export`
- from `"./persistence/index.js"` expose core type definitions and persistence helpers.
- Re-exports from `./schema.js`: `KnowledgeGraphSchema`, `validateGraph`, `sanitizeGraph`, `autoFixGraph`, `COMPLEXITY_ALIASES`, `DIRECTION_ALIASES`, `ValidationResult`, `GraphIssue`.
- Plugin exports: `TreeSitterPlugin`, `LanguageExtractor`, `builtinExtractors`, `GraphBuilder`, `PluginRegistry`.
- Analyzer exports: prompts, parsers, normalizers, search engine, staleness utilities, layer detection, tour generation, language lesson helpers.
- Language and framework config exports: `LanguageRegistry`, `FrameworkRegistry`, `builtinLanguageConfigs`, `detailedLanguageConfigs`, `builtinFrameworkConfigs`, `LanguageConfigSchema`, `FrameworkConfigSchema`, and types `LanguageConfig`, `FrameworkConfig`, `TreeSitterConfig`, `FilePatternConfig`.
- Parser exports: `MarkdownParser`, `YAMLConfigParser`, `JSONConfigParser`, `TOMLParser`, `EnvParser`, `DockerfileParser`, `SQLParser`, `GraphQLParser`, `ProtobufParser`, `TerraformParser`, `MakefileParser`, `ShellParser`, and `registerAllParsers`.
- Ignore utilities: `createIgnoreFilter`, `DEFAULT_IGNORE_PATTERNS`, `generateStarterIgnoreFile`.

Impact

- **API surface expansion:** Consumers can import any of the re-exported symbols directly from `graph-core`.
- **Build dependency:** All referenced modules must compile; missing files will break the build.
- **Type safety:** New type re-exports may surface mismatches if downstream code expects different shapes.
- **Documentation:** Update README and docs to reflect the new public exports (unknown from the available diff/scan evidence).

Risks & follow-ups

- **Missing re-exports:** Verify that every referenced module actually exists and exports the expected symbols.
- **Circular dependencies:** The large number of re-exports could introduce hidden cycles; run the linter and build to catch them.
- **Breaking changes for existing consumers:** If the package previously had a different entry point, ensure backward compatibility or provide migration guidance (unknown from the available diff/scan evidence).
- **Test coverage:** Add tests that import each re-exported symbol to confirm runtime availability and correct behavior.